

Responding to Alerts

1. Overview

Monitoring detects problems.

Alerting notifies engineers.

Incident response restores service quickly.

Alerts are only useful if they lead to fast, effective action.

Many companies fail not because systems break, but because **response is slow, unclear, or chaotic.**

2. Alert Response Lifecycle

None

Issue Happens



Monitoring Detects



Alert Triggered



On-call Engineer Responds



Diagnose Root Cause



Mitigate / Recover



Postmortem



Prevent Future Repeat

3. On-Call Engineer

Production services usually have an **on-call rotation**.

Meaning:

- One engineer owns urgent incidents for a time window
- Nights/weekends covered
- Rotates weekly or daily

Responsibilities:

- Acknowledge alerts
- Investigate incident
- Restore service quickly
- Escalate if needed

4. Severity Levels

P0 / Sev1

- Full outage
- Payment down
- Data corruption risk

Immediate response.

P1 / Sev2

- Major degradation
- High latency
- One region down

P2 / Sev3

- Minor feature issue

P3 / Sev4

- Low urgency / cosmetic

5. Runbook

A runbook is a step-by-step incident response guide.

It should include:

- Alert name
- Meaning of alert
- Possible causes
- Dashboards to check
- Commands to run
- Restart procedures
- Escalation contacts
- Rollback steps

Example

None

Alert: DB Connections > 95%

Check:

1. DB dashboard
2. Slow query panel
3. Recent deploys

Fix:

1. Restart stuck workers
2. Increase pool temporarily
3. Rollback latest release

6. Why Runbooks Matter

During incidents:

- Stress is high
- Memory is poor
- Time matters

Runbooks reduce chaos.

7. Runbook Abuse

If a runbook says:

None

1. SSH into host
2. Restart service
3. Clear temp files

and this happens often...

➡ It should be automated.

Manual repeated fixes = bad engineering practice.

8. Self-Healing Systems

Automated remediation examples:

Auto Restart

Crash detected → restart pod

Auto Scaling

CPU high → add instances

Auto Failover

Primary DB down → switch replica

Auto Rollback

New deployment causes errors → revert

9. SRE Team Role

Large companies may have **Site Reliability Engineering (SRE)** teams.

Responsibilities:

- Reliability standards
- Incident tooling
- Capacity planning
- Production readiness review
- On-call for critical systems

10. Production Readiness Checklist

Before launch:

- Good test coverage
- Dashboards ready
- Alerts configured
- Runbook written
- Capacity validated
- Rollback supported

11. During Incident: Priorities

Important interview point:

Goal is restore service first, root cause second.

Example:

Instead of debugging for 2 hours:

- Rollback deployment now
- Restore service in 5 min
- Investigate later

12. Incident Communication

Need clear communication channels.

Examples:

- Incident Slack room
- War room call
- Status page updates
- Stakeholder notifications

Roles During Incident

Role	Responsibility
Incident Commander	Coordinates response
Operator	Executes fixes
Communicator	Updates teams/customers
SME	Technical expert

13. Postmortem

After incident ends:

Write postmortem.

Include:

- What happened
- Timeline
- Impact
- Root cause
- Why detection failed
- Why recovery slow/fast
- Action items

14. Blameless Postmortem

Never focus on:

“Who caused it?”

Focus on:

“Why system/process allowed it?”

Good culture improves learning.

15. Example

Bad:

- Alice deployed bad code.

Good:

- Missing canary rollout
- No automated tests
- Alert threshold weak
- Rollback too slow

16. Metrics for Incident Response

Track:

- MTTD = Mean Time To Detect
- MTTA = Mean Time To Acknowledge
- MTTR = Mean Time To Recover
- Incident frequency

17. Common Real Incidents

- CPU spike after deploy
- Memory leak
- DB connection exhaustion
- Expired certificate
- Queue consumer stuck
- Third-party payment outage

18. Interview Script

Say:

“I’d establish on-call rotations, actionable alerts, runbooks, rollback mechanisms, and blameless postmortems. Repeated manual recovery steps should be automated into self-healing workflows.”

Excellent answer.

19. Final Summary

- Alerts must lead to action
- On-call engineers need runbooks
- Restore service first
- Automate repeated fixes
- Use postmortems to improve system
- Reliability is operational + technical

20. Memorable Line

Every alert is a test of your engineering culture.